pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Course 9 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk head's request

The request

The morning brief on a link I can send.

Say what date its figures are from.

Show the checks it passed.

Refuse anything that asks for a view.

Never hold a key.

A public repo, tests on every change.

A log of every request; a line on the cost.

Fourteen deliverables

deploykit complete; desk_app_main.py

the snapshot, hashed; the app's tests

every check empty; the repo's files

the deploy, on your accounts; a Space (optional)

monitoring; the cost line; a workbook

the description; Git with a pull request

an AI-use log

Shipped for real on the course's accounts: one public repo and one Vercel project, both pfi-desk-app.

The rules for a public app

Allowed names

the brief as of 2026-10-02

the check table

the saved run

Never called

the desk's view

advice or a call

a recommendation

Allowed names

included in Hobby as read

the free tier as read

the platform line

Never called

guaranteed

unlimited

free forever

Describe; date every response; refuse a view first; no key on the server; firm policy first.

The snapshot, rebuilt and hashed

53 / 2

the brief's checks passed and failed, as in Course 8

3

recorded answers kept: passed and sent requests

21 of 21

saved exchanges whose SHA-256 matches

```
desk_snapshot.json: 25,853 bytes, the same bytes as days 8 and 9
```

The brief's date is not the deploy date

Every figure is as of 2026-10-02 in the course's files. The app never says today and never updates a figure.

What to deploy, and the app

94

distributions to replay the desk, 923.4 MB here

11

distributions for the snapshot service, 9.6 MB

```
make_app(...):
  desk_app(snapshot, live_settings(...))
  daily_cap          # innermost
  X-Desk-As-Of       # every response
  log_middleware     # outermost
```

```
GET  /health   200 body yes  header 2026-10-02
GET  /brief    200 body yes  header 2026-10-02
GET  /checks   200 body yes  header 2026-10-02
GET  /nowhere  404 body no   header 2026-10-02
POST /ask      404 body no   header 2026-10-02
POST /ask      422 body no   header 2026-10-02
```

A cap of 2: [200, 200, 429], three log lines; the 429 is logged.

A middleware added later runs earlier. On a serverless platform the cap is a floor per process.

The app's own tests

13

passed in test_desk_app.py, no network, no key

4

requests for a view, each refused

0 / 0

names out of place in the served brief

```
200  refused: True  rule: opinion words
200  refused: True  rule: an ordering of securities
200  refused: True  rule: opinion words
200  refused: True  rule: a real issuer beside a holding
```

The tests need fastapi, httpx2, and pytest: what the workflow installs. The log never holds a question.

The deploy folder, and its checks

The starter folder

```
public_scan  1  ['desk_settings.json: key_like']

pins_check  0  []

check_deploy_folder  1  ["desk_settings.json, line 4: a k...

secret_names  2  ['DESK_MODE', 'OPENAI_API_KEY']

check_workflow  0  []
```

The planted key removed

```
public_scan  0

pins_check  0

check_deploy_folder  0

secret_names  0

check_workflow  0
```

```
app.py as an import: no entrypoint defines app
requirements.txt ['fastapi==0.143.0']
requirements-test.txt ['fastapi==0.143.0', 'httpx2==2.13.1', 'pytest==9.1.1']
```

```
README pull request:   README.md   8   0
```

The check is stricter than Vercel's page: write app = desk_app_main.app.

Ship it: GitHub

```
Set up job (34 lines)
Run actions/checkout@v7 (86 lines)
Run actions/setup-python@v7 (12 lines)
Run python -m pip install --upgrade pip (12 lines)
Run python -m pip install -r requirements-test.txt (76 ...
Run python -m pytest -q test_desk_app.py (13 lines)
Post Run actions/setup-python@v7 (1 lines)
Post Run actions/checkout@v7 (32 lines)
Complete job (1 lines)
```

```
Successfully set up CPython (3.13.16)
13 passed in 0.55s
```

```
test (3.12)   pass   14s
test (3.12)   pass   10s
test (3.13)   pass   15s
test (3.13)   pass   14s
```

gh pr merge: 1 file changed, 8 insertions(+)

```
runs listed: 4, all success; then one more: the tag's push ran the tests
```

GitHub Actions: free on public repositories, standard runners (docs.github.com, GitHub Actions billing, read 2026-10-12).

Ship it: Vercel

What the platform printed, masked

```
Recorded 2026-10-12 on the course's own account; account names, URLs, and ids masked to `student`.

> vercel deploy --prod --yes

? Created         student/pfi-desk-app

Build Completed in /vercel/output [7s]

? Aliased         https://student.vercel.app

? Ready in 25s
```

```
date 2026-10-12
target vercel production, pfi-desk-app
url_masked https://student.vercel.app
tests_passed 13
status ready
```

```
    request  status      as_of  same_as_here
GET /health     200 2026-10-02          True
 GET /brief     200 2026-10-02          True
GET /checks     200 2026-10-02          True
  POST /ask     200 2026-10-02          True
  POST /ask     404 2026-10-02          True
```

Vercel Hobby: free; non-commercial, personal use only; 100 deployments a day (vercel.com/docs/plans/hobby, read 2026-10-12).

The Space, optional

```
space_folder: 6 files;  check_space_folder: []
```

Why optional

A free account must be older than 30 days, with a verified email, and holds two ZeroGPU Spaces at most.

The required public address is Vercel's, which has no account-age rule.

The course recorded no Space today: day 10 recorded one and deleted it, keeping the slot.

Hugging Face: 2 ZeroGPU Spaces for a free account older than 30 days (huggingface.co/docs/hub/spaces-zerogpu, read 2026-10-12).

Monitoring: three logs

```
/ask          20          4          0          6    12.0    12.0
/brief        10          0          0          0    12.0    12.0
/checks       10          0          0          0    12.0    12.0
/health       10          0          0          0    12.0    12.0
```

The app's day: alert, error share 0.08; ok without the 404s.

30-day log alerts: 2026-09-17, 2026-09-24

```
13:37:59.34  /ask          404   4.6 ms
13:37:59.10  /ask          200   3.3 ms
13:37:58.88  /checks       200   1.5 ms
13:37:58.61  /brief        200   1.8 ms
13:37:58.30  /health       200   1.2 ms
13:37:56.86  /health       200   3.4 ms
13:37:54.98  /             404   0.8 ms
13:37:54.98  /favicon.ico  404   0.7 ms
13:37:54.97  /favicon.png  404   0.8 ms
```

Three requests nobody sent, all 404. A function's disk is not a log store.

Vercel Hobby: 1 hour of runtime logs; stdout is level info (vercel.com/docs/logs/runtime, read 2026-10-12).

The cost line

```
2026-09       requests 18384.000000 1000000.0       within
2026-09    transfer_gb     0.013379     100.0       within
2026-09 duration_hours     0.074961       4.0       within
2026-09      model_usd     0.000000       NaN no free tier
```

US$0

September in replay: no request to a model

US$47.91

September if the desk went live, at Course 8's prices

4

formulas in the workbook, read back as written

Vercel Hobby: 1,000,000 invocations, 4 CPU hours, 100 GB a month (vercel.com/docs/plans/hobby, read 2026-10-12).

Excel, the description, and Git

Excel side by side (tested in Excel)

`=IF(B2>C2,"over","within")` `free_tier D2:D4`	within, within, within
`=SUMPRODUCT(C2:C5,D2:D5)` `model_cost F2`	47.914342

```
description.md: 436 words; opinion_guard words found: none
8 commits, one per deliverable that writes a file; tag course9
```

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

A key-like string in any text file, of any name, reported.

→

2

Code back

Written for the course in the style of an AI answer. It runs.

→

3

Test first

A folder under Git, a tracked settings file with the fake key: assert it is found.

→

4

Fix

Drop the shortcut: the assistant's version 0; the fixed version 1

Today: a folder holding .git is declared safe and nothing is checked; Git tracks any file it is given.

Code that runs is not the same as code that is right.

What you can do now

- A snapshot whose every figure traces to a saved exchange
- A date on every response, a cap, and a log, in order
- Every check empty before anything goes public
- A public repo with CI, and a pull request read by its diff
- A public app, checked against the one on your machine

Next The Bonus: Build Your Own Coding Agent